set 1:

short question (2 marks):

1. What is vanishing gradient problem?

   - It is a problem in deep neural networks where the gradients (derivatives) become **very small (close to zero)** during backpropagation.

     Result:

     1. Earlier (hidden) layers learn **very slowly or stop learning**
     2. Weights are not updated effectively

     Cause:

     Common in deep networks using activation functions like **sigmoid** or **tanh**, where derivatives are small

     Effect:

     1. Makes training deep networks difficult
     2. Leads to poor performance or convergence failure

2. What is tensor in linear algebra? Give example.

   => A tensor is a **generalization of scalars, vectors, and matrices** to higher dimensions. It is a **multi-dimensional array of numerical values**.

   Types:

   - Scalar → 0D tensor (e.g., 5)
   - Vector → 1D tensor (e.g., [1, 2, 3])
   - Matrix → 2D tensor (e.g., [[1,2],[3,4]])
   - Higher-order tensor → 3D or more

   Example:
   A 3D tensor:
   [[[1,2],[3,4]], [[5,6],[7,8]]]

3. What is Dimension Reduction? Write the name of two methods of Dimension Reduction.

=> **Dimensionality Reduction:**
It is the process of **reducing the number of input features (variables)** in a dataset while preserving important information.

Purpose:

- Reduce complexity
- Improve model performance
- Remove redundant/noisy features

Methods (any two):

1. **Principal Component Analysis (PCA)**
2. **Linear Discriminant Analysis (LDA)**

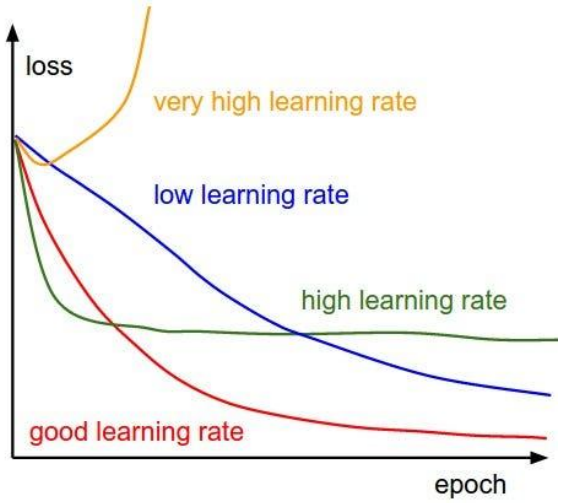
4. What happens if the learning rate is too high or too low? Explain with figure.

=> In machine learning, the **learning rate** is a hyperparameter that controls how much we adjust the model's weights in response to the estimated error each time the model is updated.

What happens if the Learning Rate is:

- **Too Low**: The model updates its parameters very slowly, leading to **slow convergence**. It requires many more training epochs to reach the minimum, which wastes computational resources. There is also a risk of the model getting **stuck in a local minimum** instead of finding the global best solution.
- **Too High**: The model makes updates that are too large, which can cause it to **overshoot** the optimal point. This leads to **oscillations** where the loss jumps up and down, or even **divergence**, where the loss increases uncontrollably instead of decreasing.
- **Optimal**: The model makes steady progress toward the minimum of the loss function, reaching it efficiently without being too slow or becoming unstable.

Visualization:



5. What is data normalization? Write the name of different techniques to achieve Data normalization.

=> **Data Normalization:**
It is the process of **scaling numerical data to a common range** so that all features contribute equally to model training.

Purpose:

- Improves convergence of models
- Prevents features with large values from dominating

Techniques:

1. **Min-Max Scaling (Normalization)**
2. **Z-score Standardization (Standard Scaling)**
3. **Decimal Scaling**
4. **Max Absolute Scaling**

6. What is activation function? Give some examples of activation functions.

=> **Activation Function:**
An activation function is a mathematical function applied to a neuron's output

to **introduce non-linearity** and decide whether the neuron should be activated or not.

Purpose:

- Helps the network learn complex patterns
- Controls output of neurons

Examples:

1. **Sigmoid**
2. **Tanh (Hyperbolic Tangent)**
3. **ReLU (Rectified Linear Unit)**
4. **Leaky ReLU**
5. **Softmax**

7. Write some application of Deep learning.

=> **Applications of Deep Learning:**

1. **Image Recognition** – object detection, face recognition
2. **Natural Language Processing (NLP)** – translation, chatbots
3. **Speech Recognition** – voice assistants
4. **Healthcare** – disease detection from medical images
5. **Autonomous Vehicles** – self-driving cars
6. **Recommendation Systems** – YouTube/Netflix suggestions

short question (2.5 marks):

1. What is the differences between ANN, CNN & RNN.

   => **Difference between ANN, CNN & RNN:**

Feature	**ANN (Artificial Neural Network)**	**CNN (Convolutional Neural Network)**	**RNN (Recurrent Neural Network)**
Structure	Fully connected layers	Convolution + pooling layers	Recurrent (loop) connections
Data Type	Works on general/tabular data	Works on image/spatial data	Works on sequential/time-series data
Memory	No memory of previous input	No memory	Has memory (previous outputs influence current)
Feature Extraction	Manual or limited	Automatic feature extraction	Learns patterns over time
Parameter Sharing	No	Yes (filters/kernels shared)	Yes (same weights reused over time)
Main Use	Classification, regression	Image recognition, computer vision	NLP, speech, time-series prediction
Example	Basic feedforward network	Image classifier (cat/dog)	Language translation, speech recognition

2. What is the differences between Dense and Sparse features in dataset?

   => **Difference between Dense and Sparse Features:**

Feature	**Dense Features**	**Sparse Features**
Definition	Most values are **non-zero and meaningful**	Most values are **zero or missing**

Feature	Dense Features	Sparse Features
Data Representation	Compact, fewer zeros	High-dimensional with many zeros
Storage	Requires more memory (store all values)	Efficient storage (store only non-zero values)
Example	Age, height, temperature	One-hot encoded categories, text data
Computation	Faster computation	Specialized methods needed for efficiency
Use Case	Numerical datasets	Categorical data, NLP, recommendation systems

Summary:
Dense features contain mostly useful values, while sparse features contain mostly zeros and are common in high-dimensional data like text or encoded categories.

3. What is the differences between Batch Gradient Descent and Stochastic Batch Gradient Descent?

=> **Difference between Batch Gradient Descent and Stochastic Gradient Descent**

Feature	Batch Gradient Descent	Stochastic Gradient Descent (SGD)
Data Used per Update	Uses **entire dataset**	Uses **one sample at a time**
Update Frequency	Updates weights once per epoch	Updates weights after every training example
Speed	Slow for large datasets	Faster and more frequent updates
Memory Requirement	High (needs full dataset in memory)	Low (only one sample needed)

Feature	Batch Gradient Descent	Stochastic Gradient Descent (SGD)
Convergence	Smooth and stable convergence	Noisy but can escape local minima
Accuracy	More stable final result	May fluctuate but often reaches good solution

Summary:
Batch Gradient Descent is stable but slow, while Stochastic Gradient Descent is fast but noisy.

4. What is Bias and Variance? What is the scenario of Bias and Variance on Model Overfitting and Underfitting?

=> **Bias and Variance**

Bias:
Bias is the error caused by **wrong or overly simple assumptions** in the learning algorithm.

- High bias means the model is too simple and does not learn the data well.

Variance:
Variance is the error caused by the model's **sensitivity to small changes in training data**.

- High variance means the model learns noise from the data.

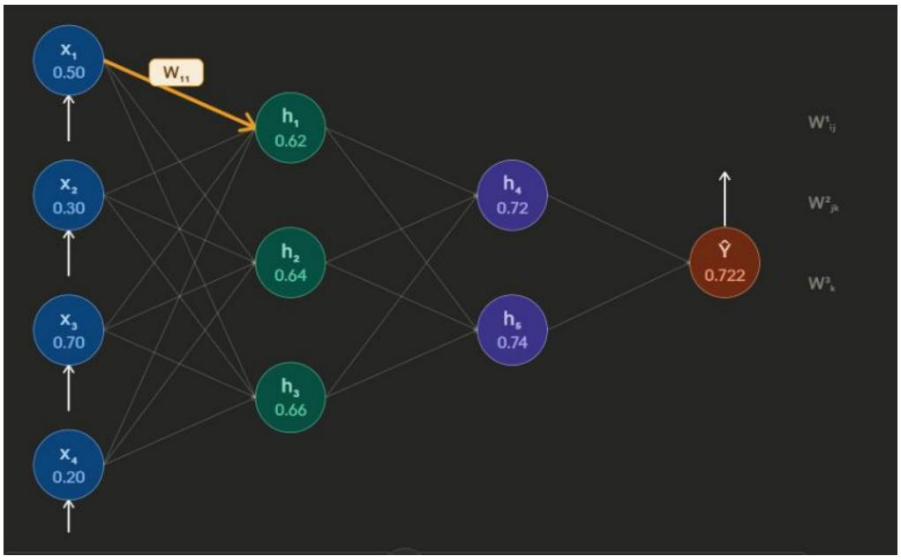
Effect on Underfitting and Overfitting

Scenario	Bias	Variance	Model Behavior
Underfitting	High	Low	Model is too simple, performs poorly on both training and test data
Overfitting	Low	High	Model learns training data too well, including noise; poor generalization
Good Model	Low	Low	Balanced model with good generalization

Summary:

- High Bias → Underfitting
- High Variance → Overfitting
- Goal → Achieve balance between bias and variance for best performance

short question (3 marks):



1.Explain the figure properly. Assume Y is not equal to Y'. Write the formula for update W11 by using chain rule equation. How will you get Y = Y'? explain your answer.

=>

1. Figure Explanation (1 Mark)

The figure shows a **Multi-Layer Perceptron (Neural Network)**.

- **Input Layer (x):** Raw data features.
- **Hidden Layers (h):** Process features using weights and activation functions.
- **Output Layer ($\hat{Y}$):** The model's prediction (0.722).
- **Weight (W_{11}):** The strength of the connection between the first input x_1 and the first hidden neuron h_1.

2. Formula for Update W_{11} (1 Mark)

We update the weight using **Gradient Descent** and the **Chain Rule** to minimize the loss (L):

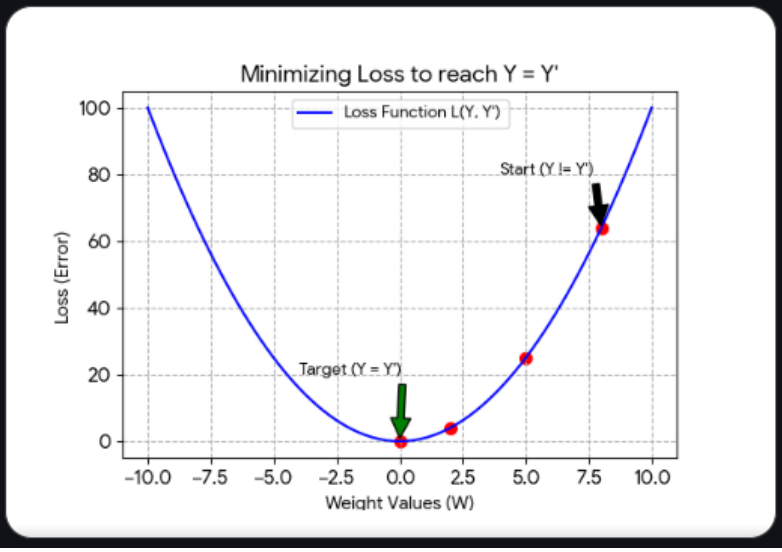
$$W_{11(new)} = W_{11(old)} - \eta \left(\frac{\partial L}{\partial Y'} \cdot \frac{\partial Y'}{\partial h_{layer2}} \cdot \frac{\partial h_{layer2}}{\partial h_1} \cdot \frac{\partial h_1}{\partial W_{11}} \right)$$

- η is the **learning rate**.
- The term in parentheses represents the gradient $\frac{\partial L}{\partial W_{11}}$.

3. How to get $Y = Y'$? (1 Mark)

To reach $Y = Y'$ (zero error), the network undergoes **Iterative Training**:

1. **Backpropagation:** Calculate the error (Loss) between Y and Y'.
2. **Optimizer:** Use the gradient calculated above to adjust all weights (W) in the direction that reduces the Loss.
3. **Convergence:** Repeat this process over many epochs until the weights are optimized such that the prediction Y' perfectly matches the target Y.



2. How does CNN works? Explain with proper diagram.

=> A CNN works by automatically extracting features from an input image and then using them for classification.

Working Steps

1. **Input Layer**

   Takes an image as input (pixel matrix)

2. **Convolution Layer**

   - Applies **filters/kernels**
   - Extracts features like edges, corners, textures
   - Produces **feature maps**

3. **Activation (ReLU)**

   - Introduces non-linearity
   - Removes negative values

4. **Pooling Layer**

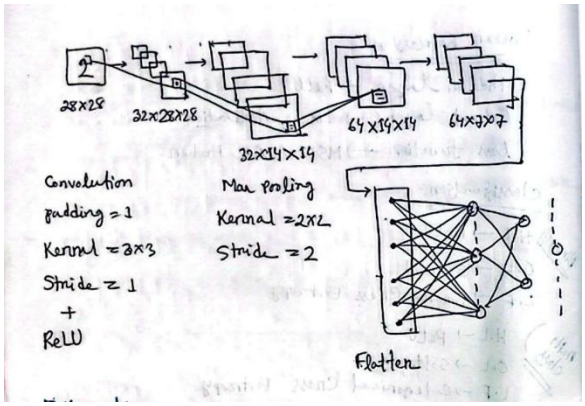
   - Reduces feature map size
   - Keeps important features (Max/Avg pooling)

5. **Fully Connected Layer**

   - Flattens data into 1D vector
   - Connects to neural network for classification

6. **Output Layer**

   Gives final prediction (e.g., Cat / Dog)



Summary:

CNN extracts features step-by-step using convolution and pooling, then classifies the image using fully connected layers.

3. Describe the architecture of LSTM with proper diagram.

=> **Architecture of LSTM (Long Short-Term Memory)**

LSTM is a special type of **Recurrent Neural Network (RNN)** designed to handle long-term dependencies by using a **memory cell** and different **gates** to control information flow.

Main Components of LSTM:

1. Cell State (Ct)

- Acts as the **memory of the network**
- Carries important information across time steps

2. Hidden State (ht)

- Output of the LSTM at each time step
- Used for prediction and passed to next step

3 Gates of LSTM:

1. Forget Gate (ft)

- Decides what information to **remove** from cell state
- Formula idea: uses sigmoid activation (0 to 1)

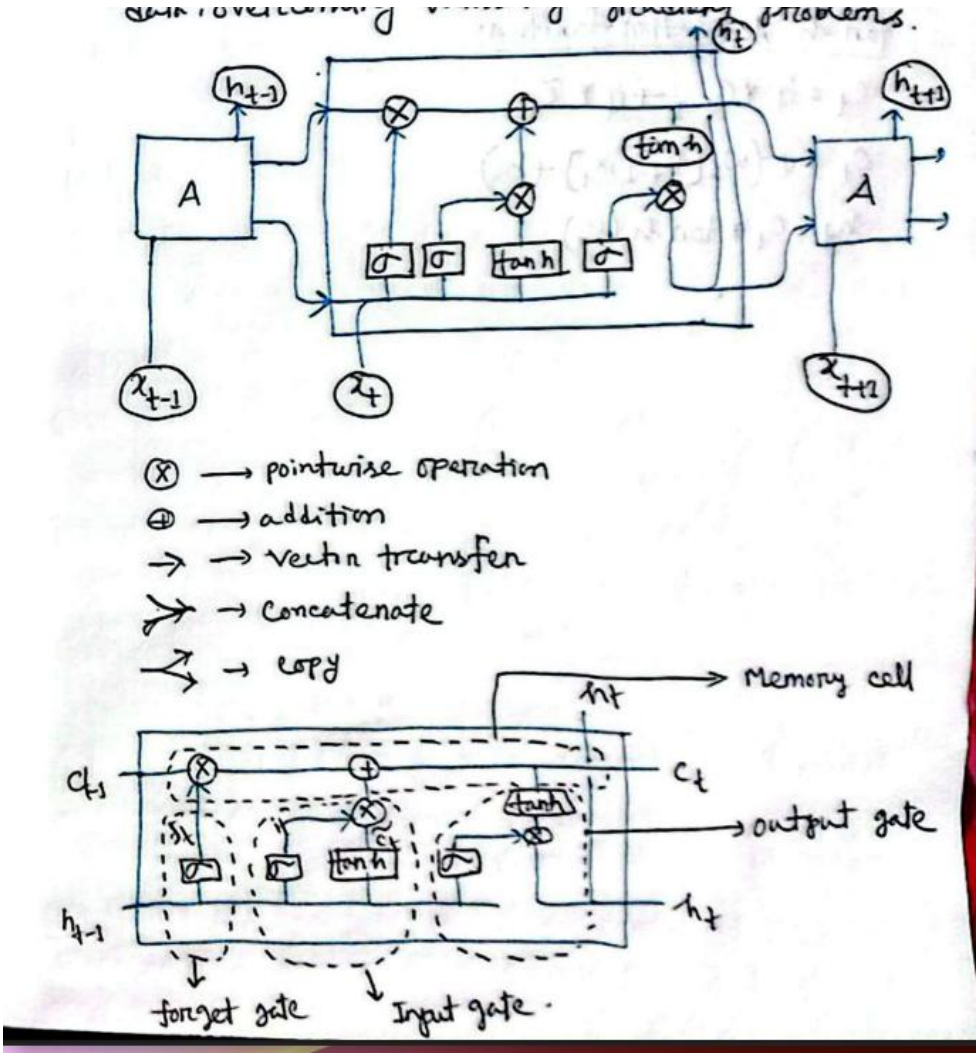
2. Input Gate (it)

- Decides what new information to **store** in memory

3. Output Gate (ot)

- Controls what part of cell state becomes the **output**

Simple LSTM Flow Diagram:



Summary:

LSTM architecture uses **cell state + three gates (forget, input, output)** to control information flow and solve the vanishing gradient problem in RNNs.

4. What is perceptron? Write the perceptron learning algorithm. Explain the geometric view of perceptron.

=>

Perceptron

A **Perceptron** is a **supervised learning algorithm** used for **binary classification**. It is the simplest type of artificial neural network that models a single neuron.

It computes a weighted sum of inputs and passes it through an activation function:

$$y = \text{sign}(w \cdot x + b)$$

Where:

- w = weight vector
- x = input vector
- b = bias
- Output $y \in \{-1, +1\}$

Perceptron Learning Algorithm

Steps:

1. Initialize weights w and bias b (usually small random values or zeros).
2. For each training example (x_i, y_i):
   - Compute output:

$$\hat{y} = \text{sign}(w \cdot x_i + b)$$

   - Update weights if misclassified:

$$w = w + \eta\, y_i x_i$$

$$b = b + \eta\, y_i$$

3. Repeat until:
   - All points are correctly classified OR
   - Maximum iterations reached

Where η = learning rate.

Geometric View of Perceptron

- The perceptron learns a **linear decision boundary (hyperplane)**:

$$w \cdot x + b = 0$$

- In:
  - **2D** → a straight line
  - **3D** → a plane
  - **Higher dimensions** → a hyperplane

Key Idea:

- The hyperplane **divides the feature space into two regions**:
  - One side → class +1
  - Other side → class −1

Interpretation:

- w is **perpendicular (normal)** to the decision boundary
- b shifts the boundary
- During training, the perceptron **adjusts the boundary** to correctly separate classes

Important Note:

- Works only for **linearly separable data**

set 2

short question (2 marks):

1. What are the main applications of deep learning?

=> **Main Applications of Deep Learning (2 marks)**

- **Computer Vision:** Image classification, object detection, face recognition
- **Natural Language Processing (NLP):** Machine translation, chatbots, sentiment analysis
- **Speech Recognition:** Voice assistants and speech-to-text systems
- **Healthcare:** Disease diagnosis, medical image analysis
- **Autonomous Systems:** Self-driving cars and robotics

In short: Deep learning is widely used in vision, language, speech, healthcare, and automation.

2. What is data normalization? Write the name of different techniques to achieve Data normalization.

=> Data normalization refers to the process of scaling or adjusting data so that its range fits within a specified range, typically between 0 and 1 or -1 and 1. This ensures that features are on a comparable scale and reduces the impact of large differences in input values.

In the context of deep learning, normalization techniques are used during training to improve model performance by addressing issues like internal covariate shift. Some key techniques include:

1. Standardization (Z-score normalization): Scales data so that it has zero mean and unit variance.
2. Scaling to [0, 1]: Rescaling values between 0 and 1 using min-max scaling.
3. Batch Normalization: A technique where each batch of samples is normalized at training time by computing statistics like mean and variance.

These techniques help in training deep neural networks more effectively by normalizing the input features to a consistent range.

3. How to remove Vanishing Gradient Problem?

=> The Vanishing Gradient Problem is a challenge encountered during the training of deep neural networks where the gradients of the loss function with respect to the weights become vanishingly small, slowing down or halting learning. This problem can be addressed through several strategies:

1. Activation Functions: Using activation functions that maintain large gradients, such as ReLU (Rectified Linear Unit), Leaky ReLU, and sigmoid variants, helps mitigate the issue by preventing gradient decay early in the network.
2. Normalization Techniques: Methods like batch normalization or layer normalization scale and shift the outputs of each layer independently to maintain stable gradients across all layers. This helps prevent internal covariate shift and keeps the activation functions active throughout the training process.
3. Optimization Algorithms: Some optimization techniques, such as Adam optimizer with a composite learning rate schedule, are designed to handle gradient scaling better by adjusting the learning rates based on the magnitudes of the parameters.
4. Residual Connections: Introducing skip connections that bypass certain layers during training can help prevent gradients from dying entirely by allowing gradients to flow through the network.

Each strategy has its own benefits and may be more effective depending on the specific context, architecture, and resources available.

4. Write three differences between ANN, CNN & RNN.

=> **Differences between ANN, CNN & RNN**

Feature	**ANN (Artificial Neural Network)**	**CNN (Convolutional Neural Network)**	**RNN (Recurrent Neural Network)**
Data Type	Works on structured/tabular data	Works on image/spatial data	Works on sequential/time-series data

Feature	ANN (Artificial Neural Network)	CNN (Convolutional Neural Network)	RNN (Recurrent Neural Network)
Architecture	Fully connected layers	Convolution + pooling layers	Has feedback/recurrent connections
Memory/Context	No memory of previous input	Captures spatial features	Remembers previous inputs (has memory)

Short note:
ANN = general purpose, CNN = spatial feature extraction, RNN = sequence learning.

5. WHAT ARE GLOBAL MINIMA AND LOCAL MINIMA?

=> **Global Minima vs Local Minima**

- **Global Minima:**
  The **lowest point** of the loss function over the entire space. It gives the **best possible solution**.
- **Local Minima:**
  A point where the loss is lower than nearby points but **not the lowest overall**. It may lead to a **suboptimal solution**.

In short:
Global minima = absolute best solution,
Local minima = locally best but not optimal overall.

6. WHEN WE CALCULATE MEAN SQUARED ERROR(MSE) AND BINARY CROSS ENTROPY? WRITE THE EQUATIONS OF THESE LOSS FUNCTION.

=>

MSE and Binary Cross Entropy (2–3 marks)

When to use:

- **Mean Squared Error (MSE):**
  Used for **regression problems** (continuous values)
- **Binary Cross Entropy (BCE):**
  Used for **binary classification problems** (output 0 or 1)

Equations:

1. Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Where:

- y_i = actual value
- $\hat{y}_i$ = predicted value

---

2. Binary Cross Entropy (BCE):

$$\text{BCE} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

7. Write the equation for ReLU activation function. Give two advantages of ReLU activation function.

=>

ReLU Activation Function (2 marks)

Equation:

$$f(x) = \max(0, x)$$

Advantages (any two):

- **Computationally efficient**: Simple function, speeds up training
- **Reduces vanishing gradient problem**: Helps deep networks learn better

Short note: ReLU outputs 0 for negative inputs and the same value for positive inputs.

short question (2.5 marks):

1. Describe PCA(Principle Component Analysis) with an example.

=> PCA is a **dimensionality reduction technique** that transforms high-dimensional data into a **lower-dimensional space** while preserving maximum variance.

It finds new variables called **principal components**, which are uncorrelated.

Example:
If a dataset has features like **height, weight, age**, PCA may combine them into one or two components representing overall **body characteristics**, reducing dimensions while keeping important information.

2. What is Saturating Function? Explain it by two activation functions.

=>

2. Saturating Function

A **saturating function** is an activation function whose output becomes **nearly constant for very large or very small inputs**, causing gradients to become very small.

Examples:

- **Sigmoid**:

$$f(x) = \frac{1}{1 + e^{-x}}$$

Output ranges between **0 and 1**, saturates near 0 and 1.

- **Tanh**:

$$f(x) = \tanh(x)$$

Output ranges between **−1 and 1**, saturates at extremes.

3. Explain Adagrad Optimizer.

=>

3. Adagrad Optimizer

Adagrad is an optimizer that **adapts learning rate for each parameter** based on past gradients.

Idea:

- Frequently updated parameters → smaller learning rate
- Rare parameters → larger learning rate

Update rule:

$$w = w - \frac{\eta}{\sqrt{G + \epsilon}} \cdot g$$

Where G = sum of squared gradients.

Advantage: Works well for sparse data
Disadvantage: Learning rate decreases too much over time

4. Assume you have a neuron with two inputs X = [3,7] that uses both the ReLU and Sigmoid activation functions separately.

The neuron has the following parameters:

•Weights = [0.4, 0.6] • Bias = 0.2

Calculate the output of the neuron during the feedforward process.

=>

4. Feedforward Calculation (ReLU & Sigmoid)

Given:

- $X = [3, 7]$
- $W = [0.4, 0.6], b = 0.2$

Step 1: Weighted sum

$$z = (3 \times 0.4) + (7 \times 0.6) + 0.2 = 1.2 + 4.2 + 0.2 = 5.6$$

(a) ReLU output

$$f(z) = \max(0, 5.6) = 5.6$$

(b) Sigmoid output

$$f(z) = \frac{1}{1 + e^{-5.6}} \approx 0.996$$

Final Answer:

- ReLU output = **5.6**
- Sigmoid output ≈ **0.996**

question (5 marks):

1. How does CNN work? Explain with proper diagram.

=>

=> A CNN works by automatically extracting features from an input image and then using them for classification.

Working Steps

1. **Input Layer**

   Takes an image as input (pixel matrix)

2. **Convolution Layer**

   - Applies **filters/kernels**
   - Extracts features like edges, corners, textures
   - Produces **feature maps**

3. **Activation (ReLU)**

   - Introduces non-linearity
   - Removes negative values

4. **Pooling Layer**

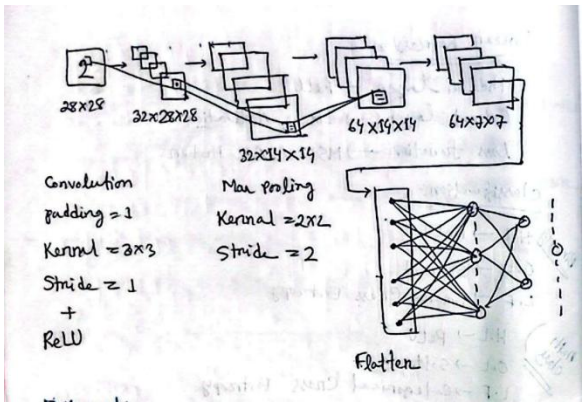
   - Reduces feature map size
   - Keeps important features (Max/Avg pooling)

5. **Fully Connected Layer**

   - Flattens data into 1D vector
   - Connects to neural network for classification

6. **Output Layer**

   Gives final prediction (e.g., Cat / Dog)



Summary:

CNN extracts features step-by-step using convolution and pooling, then classifies the image using fully connected layers.

2. Describe the architecture of LSTM with proper diagram.

=>

Architecture of LSTM (Long Short-Term Memory)

LSTM is a special type of **Recurrent Neural Network (RNN)** designed to handle long-term dependencies by using a **memory cell** and different **gates** to control information flow.

Main Components of LSTM:

1. Cell State (Ct)

- Acts as the **memory of the network**
- Carries important information across time steps

2. Hidden State (ht)

- Output of the LSTM at each time step
- Used for prediction and passed to next step

3 Gates of LSTM:

1. Forget Gate (ft)

- Decides what information to **remove** from cell state
- Formula idea: uses sigmoid activation (0 to 1)

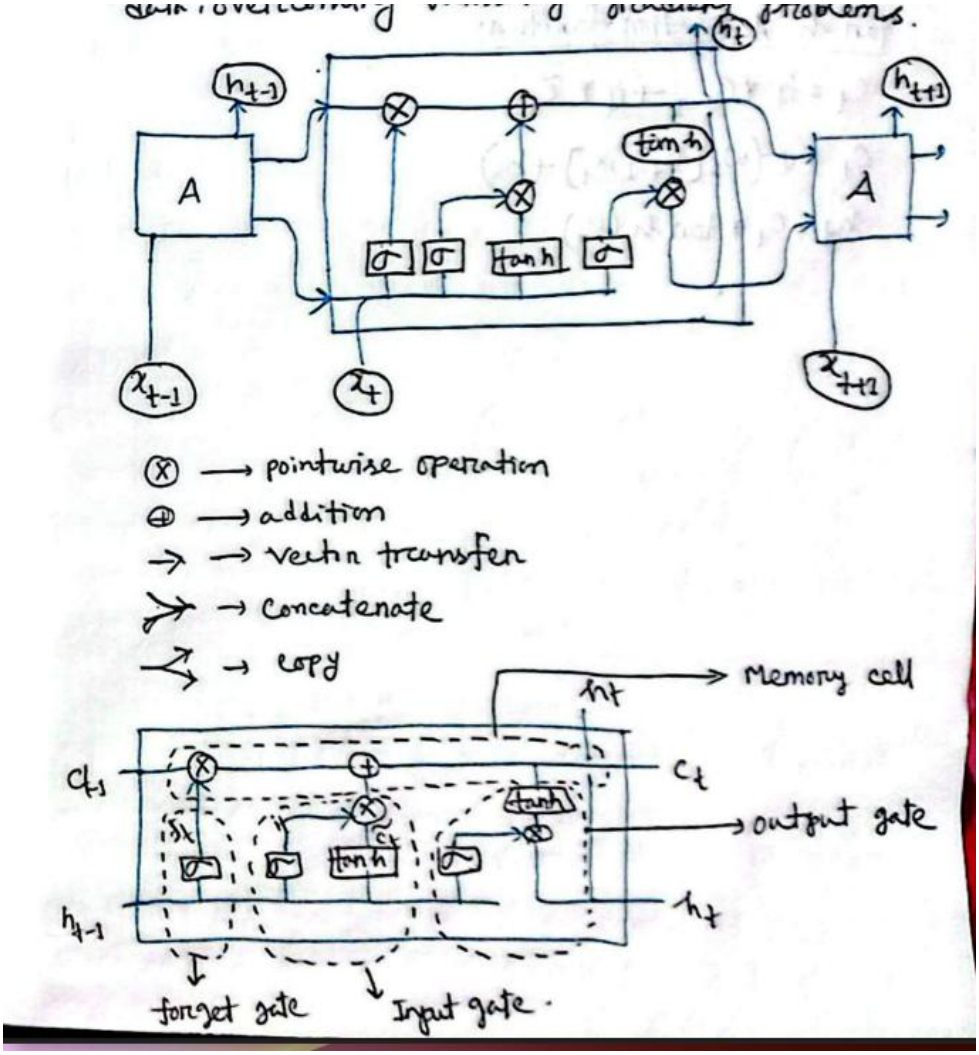
2. Input Gate (it)

- Decides what new information to **store** in memory

3. Output Gate (ot)

- Controls what part of cell state becomes the **output**

Simple LSTM Flow Diagram:



Summary:

LSTM architecture uses **cell state + three gates (forget, input, output)** to control information flow and solve the vanishing gradient problem in RNNs.

3. What is perceptron? Write the perceptron learning algorithm. Explain the geometric view of perceptron.

=>

Perceptron

A **Perceptron** is a **supervised learning algorithm** used for **binary classification**. It is the simplest type of artificial neural network that models a single neuron.

It computes a weighted sum of inputs and passes it through an activation function:

$$y = \text{sign}(w \cdot x + b)$$

Where:

- w = weight vector
- x = input vector
- b = bias
- Output $y \in \{-1, +1\}$

Perceptron Learning Algorithm

Steps:

1. Initialize weights w and bias b (usually small random values or zeros).
2. For each training example (x_i, y_i):
   - Compute output:

$$\hat{y} = \text{sign}(w \cdot x_i + b)$$

   - Update weights if misclassified:

$$w = w + \eta\, y_i x_i$$

$$b = b + \eta\, y_i$$

3. Repeat until:
   - All points are correctly classified OR
   - Maximum iterations reached

Where η = learning rate.

Geometric View of Perceptron

- The perceptron learns a **linear decision boundary (hyperplane)**:

$$w \cdot x + b = 0$$

- In:
  - **2D** → a straight line
  - **3D** → a plane
  - **Higher dimensions** → a hyperplane

Key Idea:

- The hyperplane **divides the feature space into two regions**:
  - One side → class +1
  - Other side → class −1

Interpretation:

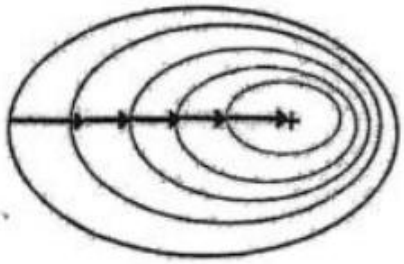
- w is **perpendicular (normal)** to the decision boundary
- b shifts the boundary
- During training, the perceptron **adjusts the boundary** to correctly separate classes

Important Note:

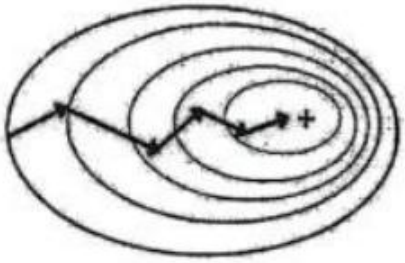
- Works only for **linearly separable data**

4.

1) Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent

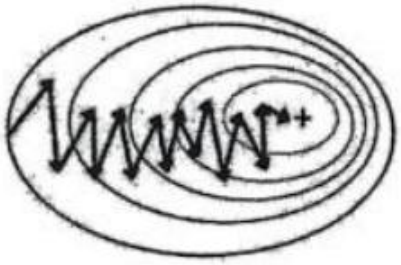


Figure: Optimizers

What is Noise in optimizer? How do you reduce the Noise? Explain with mathematical formula.

=>

🔷 What is *Noise* in an Optimizer?

In optimization (especially **Stochastic Gradient Descent - SGD**), *noise* refers to the randomness in gradient estimates when we compute gradients using a **subset of data (mini-batch or single sample)** instead of the full dataset.

Mathematically:

- True gradient (full batch):

$$\nabla J(\theta) = \frac{1}{N} \sum_{i=1}^{N} \nabla L_i(\theta)$$

- Stochastic / mini-batch gradient:

$$\tilde{\nabla} J(\theta) = \frac{1}{|B|} \sum_{i \in B} \nabla L_i(\theta)$$

Here,
$\tilde{\nabla} J(\theta)$ is an **approximation** of the true gradient
→ this difference introduces **noise**.

So,

↓

$$\text{Noise} = \tilde{\nabla} J(\theta) - \nabla J(\theta)$$

🔷 Why Noise Occurs?

- Using **random samples (mini-batch / single sample)**
- Data variability
- Leads to **zig-zag path** (as shown in SGD diagram)

🔷 How to Reduce Noise?

✅ 1. Increase Batch Size

Larger batch → better approximation:

$$\text{Var}(\tilde{\nabla} J) \propto \frac{1}{|B|}$$

👉 As batch size $|B|$ increases → variance (noise) decreases.

✅ 2. Use Mini-Batch Instead of Pure SGD

Instead of 1 sample:

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{|B|} \sum_{i \in B} \nabla L_i(\theta_t)$$

✅ 3. Learning Rate Decay

Reduce step size over time:

$$\eta_t = \frac{\eta_0}{1 + kt}$$

👉 Smaller steps → less oscillation (noise effect reduced)

✅ 4. Momentum (Smooth Updates)

$$v_t = \beta v_{t-1} + (1 - \beta)\nabla J(\theta_t)$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

👉 Averages gradients → reduces noise

✅ **5. Advanced Optimizers (Adam, RMSProp)**
Example (Adam):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla J(\theta)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla J(\theta))^2$$

👉 Adaptive smoothing reduces noise significantly

🔷 **Summary**

- Noise = error in gradient estimation due to sampling
- High in SGD, lower in Mini-batch, lowest in Batch GD
- Reduced by:
  - Increasing batch size
  - Momentum
  - Learning rate decay
  - Adaptive optimizers